Lecture 31: MIPS Stack Frames & Calling Conventions (Advanced)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand what a stack frame is.
- Explain how procedure calls use stack frames.
- Differentiate between caller-saved and callee-saved registers.
- Save and restore registers correctly.
- Build nested procedure calls.
- Understand stack layout during execution.
- Debug stack-related problems in the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Mention naturally:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how stack frames work in MIPS Assembly Language, understand the MIPS calling convention, explore caller-saved and callee-saved registers, and build nested procedures using the MARS simulator."**

In the previous lecture, we learned how recursion works and why recursive procedures require the stack.

Today, we'll go one step further and learn how professional Assembly programs manage procedure calls using stack frames and the MIPS calling convention.

Whenever a function is called, the processor must remember important information such as the return address, parameters, local variables, and saved registers. All of this is organized inside a stack frame.

Understanding stack frames is essential for writing reliable Assembly programs and is one of the most frequently tested topics in COAL courses.

Let's begin.

---

2. Revision – What is the Stack?

The **stack** is a memory area that stores temporary information during program execution.

It follows the **LIFO (Last In, First Out)** principle.

Example:

Push A

Push B

Push C

Pop C

Pop B

Pop A

The last item pushed onto the stack is the first one removed.

---

3. What is a Stack Frame?

A **stack frame** is the portion of the stack reserved for one procedure call.

A stack frame typically stores:

- Return address (`$ra`)
- Saved registers (`$s0–$s7`)
- Local variables
- Temporary data

Visualization:

Higher Memory

_____

Previous Stack Frame

_____

Saved $ra

Saved $s0

Local Variable 1

Local Variable 2

_____

← $sp

Lower Memory

Each procedure creates its own stack frame.

---

4. Why Do We Need Stack Frames?

Suppose:

main()

↓

Function A()

↓

Function B()

↓

Function C()

Each function needs its own:

- Return address
- Saved registers
- Local data

Without stack frames, functions would overwrite each other's data.

---

5. Creating a Stack Frame

Example:

addi $sp,$sp,-12

sw $ra,8($sp)

sw $s0,4($sp)

sw $s1,0($sp)

Explanation:

- Reserve **12 bytes**.
- Save `$ra`.
- Save `$s0`.
- Save `$s1`.

---

6. Restoring a Stack Frame

Before returning:

lw $s1,0($sp)

lw $s0,4($sp)

lw $ra,8($sp)

addi $sp,$sp,12

jr $ra

Always restore registers in the reverse order of how they were saved.

---

7. Caller-Saved vs Callee-Saved Registers

Caller-Saved Registers

These include:

$t0 – $t9

If the caller needs their values after a procedure call, it must save them before calling another function.

Callee-Saved Registers

These include:

$s0 – $s7

If a procedure modifies these registers, it must save and restore them before returning.

---

8. Standard MIPS Calling Convention

Register	Purpose
`$a0–$a3`	Function arguments
`$v0–$v1`	Return values
`$t0–$t9`	Temporary (caller-saved)
`$s0–$s7`	Saved (callee-saved)
`$ra`	Return address
`$sp`	Stack pointer

Stress that following this convention makes different procedures compatible.

---

9. Example – Nested Procedure Calls

main()

↓

Add()

↓

Multiply()

↓

Display()

Flow:

1. `main` calls `Add`.
2. `Add` creates its stack frame.
3. `Add` calls `Multiply`.
4. `Multiply` creates another stack frame.
5. `Multiply` returns.
6. `Add` returns.
7. `main` continues.

---

10. Visualizing Multiple Stack Frames

Higher Memory

---

main()

Add()

Saved $ra

Saved $s0

Multiply()

Saved $ra

Saved $s0

← $sp

Lower Memory

Every function has its own frame.

---

11. Local Variables on the Stack

Sometimes a function needs local storage.

Example:

addi $sp,$sp,-16

Possible layout:

12($sp) → Saved $ra

8($sp) → Saved $s0

4($sp) → Local Variable

0($sp) → Local Variable

This avoids using unnecessary registers.

---

12. Complete Example

Function:

addi $sp,$sp,-8

sw $ra,4($sp)

sw $s0,0($sp)

Procedure Body

lw $s0,0($sp)

lw $ra,4($sp)

addi $sp,$sp,8

jr $ra

Explain every instruction and why it is needed.

---

13. Debugging Stack Frames in MARS

Demonstrate:

1. Assemble the program.
2. Set a breakpoint before the procedure call.
3. Step through execution.
4. Observe:
    - `$sp`
    - `$ra`
    - `$s0`
5. Watch the stack grow downward and shrink as functions return.

This helps students connect the code to the memory layout.

---

14. Real-Life Analogy

Imagine a hotel.

Each guest (procedure) receives a separate room (stack frame).

Inside the room:

- Personal belongings (local variables)
- Room key (return address)
- Important documents (saved registers)

When the guest checks out, the room is cleaned and becomes available for the next guest.

---

15. Common Mistakes

Students often:

❌ Forget to save `$ra` before calling another procedure.

❌ Modify `$s0–$s7` without restoring them.

❌ Allocate stack space but forget to release it.

Example:

addi $sp,$sp,-8

must always be matched with:

addi $sp,$sp,8

❌ Restore registers in the wrong order or use incorrect offsets.

Always keep the save and restore offsets consistent.

🎤 Lecture Summary

Today, we explored advanced stack management in MIPS Assembly Language.

We learned what stack frames are, why they are required for procedure calls, how to save and restore registers correctly, and how the MIPS calling convention organizes procedure execution. We also examined nested procedure calls and visualized stack growth and shrinkage using the MARS simulator.

Thank you for watching, and I'll see you in the next course.
